

Name:- Shivasai Thodeti

Email:-shivasai591@gmail.com

Mobile:-6309342042

Batch Name:-B1_Python

Course:UGE_DoNet FSD (Python)

=====

1.Project :

Models/users –

using System.ComponentModel.DataAnnotations;

namespace SmartSkillExchange.Models

```
{  
    public class User  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string Name { get; set; }  
  
        [Required, EmailAddress]  
        public string Email { get; set; }  
  
        public List<Skill> Skills { get; set; }  
        public Wallet Wallet { get; set; }  
    }  
}
```

Models/Skills.cs :

using System.ComponentModel.DataAnnotations;

```

namespace SmartSkillExchange.Models
{
    public class Skill
    {
        public int Id { get; set; }

        [Required]
        public string Title { get; set; }

        [Required]
        public string Description { get; set; }

        public int UserId { get; set; }
        public User User { get; set; }

        public List<Booking> Bookings { get; set; }
    }
}

```

Models/Bookings.cs :

```

namespace SmartSkillExchange.Models
{
    public class Booking
    {
        public int Id { get; set; }

        public int SkillId { get; set; }
        public Skill Skill { get; set; }

        public int UserId { get; set; }
    }
}

```

```

        public User User { get; set; }
    }
}

Models/Reviews.cs –
namespace SmartSkillExchange.Models
{
    public class Review
    {
        public int Id { get; set; }
        public string Comment { get; set; }
        public int Rating { get; set; }

        public int SkillId { get; set; }
        public Skill Skill { get; set; }
    }
}

```

```

Models/wallets.cs –
namespace SmartSkillExchange.Models
{
    public class Wallet
    {
        public int Id { get; set; }
        public int Points { get; set; } = 100;

        public int UserId { get; set; }
        public User User { get; set; }
    }
}

```

DB CONTEXT :

```

using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Models;

namespace SmartSkillExchange.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
        public DbSet<Skill> Skills { get; set; }
        public DbSet<Booking> Bookings { get; set; }
        public DbSet<Review> Reviews { get; set; }
        public DbSet<Wallet> Wallets { get; set; }
    }
}

```

Program.cs –

```

using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

```

```
builder.Services.AddSession();
```

```
var app = builder.Build();
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseSession();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Account}/{action=Login}/{id?}");
```

```
app.Run();
```

Account Controller :

```
using Microsoft.AspNetCore.Mvc;
```

```
using SmartSkillExchange.Data;
```

```
using SmartSkillExchange.Models;
```

```
namespace SmartSkillExchange.Controllers
```

```
{
```

```
    public class AccountController : Controller
```

```
    {
```

```
        private readonly ApplicationDbContext _context;
```

```
        public AccountController(ApplicationDbContext context)
```

```
        {
```

```
            _context = context;
```

```
}
```

```
public IActionResult Register()
```

```
{
```

```
    return View();
```

```
}
```

```
[HttpPost]
```

```
public IActionResult Register(User user)
```

```
{
```

```
    if (ModelState.IsValid)
```

```
    {
```

```
        _context.Users.Add(user);
```

```
        _context.SaveChanges();
```

```
        // create wallet
```

```
        var wallet = new Wallet { UserId = user.Id };
```

```
        _context.Wallets.Add(wallet);
```

```
        _context.SaveChanges();
```

```
        return RedirectToAction("Login");
```

```
    }
```

```
    return View(user);
```

```
}
```

```
public IActionResult Login()
```

```
{
```

```
    return View();
```

```
}
```

```

[HttpPost]

public IActionResult Login(string email)
{
    var user = _context.Users.FirstOrDefault(u => u.Email == email);

    if (user != null)
    {
        HttpContext.Session.SetInt32("UserId", user.Id);
        return RedirectToAction("Index", "Skill");
    }

    ViewBag.Error = "Invalid login";
    return View();
}
}
}

```

Skill Controller :

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Data;
using SmartSkillExchange.Models;

namespace SmartSkillExchange.Controllers
{
    public class SkillController : Controller
    {
        private readonly ApplicationDbContext _context;
    }
}

```

```
public SkillController(ApplicationDbContext context)
{
    _context = context;
}
```

```
public IActionResult Index()
{
    var skills = _context.Skills.Include(s => s.User).ToList();
    return View(skills);
}
```

```
public IActionResult Create()
{
    return View();
}
```

```
[HttpPost]
public IActionResult Create(Skill skill)
{
    var userId = HttpContext.Session.GetInt32("UserId");

    if (userId == null)
        return RedirectToAction("Login", "Account");

    if (ModelState.IsValid)
    {
        skill.UserId = userId.Value;
        _context.Skills.Add(skill);
        _context.SaveChanges();
    }
}
```



```
        TempData["Success"] = "Skill added!";  
        return RedirectToAction("Index");  
    }  
  
    return View(skill);  
}
```

```
public IActionResult Edit(int id)  
{  
    var skill = _context.Skills.Find(id);  
    return View(skill);  
}
```

```
[HttpPost]  
public IActionResult Edit(Skill skill)  
{  
    if (ModelState.IsValid)  
    {  
        _context.Skills.Update(skill);  
        _context.SaveChanges();  
        return RedirectToAction("Index");  
    }  
    return View(skill);  
}
```

```
public IActionResult Delete(int id)  
{  
    var skill = _context.Skills.Find(id);
```

```

        _context.Skills.Remove(skill);
        _context.SaveChanges();
        return RedirectToAction("Index");
    }
}

```

Booking Controller :

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Data;
using SmartSkillExchange.Models;

namespace SmartSkillExchange.Controllers
{
    public class BookingController : Controller
    {
        private readonly ApplicationDbContext _context;

        public BookingController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Create(int skillId)
        {
            var skill = _context.Skills.Find(skillId);
            return View(skill);
        }
    }
}

```

```
[HttpPost]

public IActionResult CreateBooking(int skillId)
{
    var userId = HttpContext.Session.GetInt32("UserId");

    var wallet = _context.Wallets.FirstOrDefault(w => w.UserId == userId);

    if (wallet.Points < 10)
    {
        TempData["Error"] = "Not enough points!";
        return RedirectToAction("Index", "Skill");
    }

    wallet.Points -= 10;

    var booking = new Booking
    {
        SkillId = skillId,
        UserId = userId.Value
    };

    _context.Bookings.Add(booking);
    _context.SaveChanges();

    TempData["Success"] = "Booking successful!";
    return RedirectToAction("Index", "Skill");
}
}
```

SIMPLE VIEWS :

```
@model List<SmartSkillExchange.Models.Skill>
```

```
<h2>Skills</h2>
```

```
@if (TempData["Success"] != null)
```

```
{
```

```
    <p>@TempData["Success"]</p>
```

```
}
```

```
<a href="/Skill/Create">Add Skill</a>
```

```
@foreach (var item in Model)
```

```
{
```

```
    <div>
```

```
        <h3>@item.Title</h3>
```

```
        <p>@item.Description</p>
```

```
        <p>By: @item.User.Name</p>
```

```
        <a href="/Booking/Create?skillId=@item.Id">Book</a>
```

```
        <a href="/Skill/Edit/@item.Id">Edit</a>
```

```
        <a href="/Skill/Delete/@item.Id">Delete</a>
```

```
    </div>
```

```
}
```

```
@model SmartSkillExchange.Models.Skill
```

```
<form asp-action="Create" method="post">
```

```
    <input asp-for="Title" placeholder="Title" />
```

```
    <input asp-for="Description" placeholder="Description" />
```

```
<button type="submit">Save</button>
</form>
```

Views/Accounts :

```
<form method="post">
    <input name="email" placeholder="Enter email" />
    <button type="submit">Login</button>
</form>
```

```
<p>@ViewBag.Error</p>
```

Views/Account/Register.cshtml :

```
@model SmartSkillExchange.Models.User
```

```
<form method="post">
    <input asp-for="Name" placeholder="Name" />
    <input asp-for="Email" placeholder="Email" />
    <button type="submit">Register</button>
</form>
```

Views/Accounts.html :

```
@model SmartSkillExchange.Models.User
```

```
<form method="post">
    <input asp-for="Name" placeholder="Name" />
    <input asp-for="Email" placeholder="Email" />
    <button type="submit">Register</button>
</form>
```

Connection String :

```
"ConnectionStrings": {
    "DefaultConnection":
    "Server=.;Database=SmartSkillDb;Trusted_Connection=True;TrustServerCertificate=True;"
```

```
}
```

Migrations Commands :

Date:- 24/03/2026 & 25/03/2026

=====

MODELS.CS –

namespace OrderApi.Models

```
{
```

```
    public enum OrderStatus
```

```
    {
```

```
        Pending,
```

```
        Completed,
```

```
        Cancelled,
```

```
        HighValue
```

```
    }
```

```
    public class Order
```

```
    {
```

```
        public int Id { get; set; }
```

```
        public string CustomerName { get; set; }
```

```
        public double TotalAmount { get; set; }
```

```
        public OrderStatus Status { get; set; }
```

```
    }
```

```
}
```

DTO –

namespace OrderApi.DTOs

```
{
```

```
public class OrderDto
{
    public string CustomerName { get; set; }
    public double TotalAmount { get; set; }
}
}
```

DB CONTEXT :

```
using Microsoft.EntityFrameworkCore;
using OrderApi.Models;
```

```
namespace OrderApi.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Order> Orders { get; set; }
    }
}
```

REPOSITORY INTERFACE :

```
using OrderApi.Models;
```

```
namespace OrderApi.Repositories
{
    public interface IOrderRepository
    {

```

```

        Task<List<Order>> GetAllOrders();

        Task<Order?> GetOrderById(int id);

        Task<Order> CreateOrder(Order order);

        Task<bool> UpdateStatus(int id, string status);

        Task<bool> DeleteOrder(int id);
    }
}

```

REPOSITORY IMPLEMENTATION :

```

using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Models;

namespace OrderApi.Repositories
{
    public class OrderRepository : IOrderRepository
    {
        private readonly AppDbContext _context;

        public OrderRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<List<Order>> GetAllOrders()
        {
            return await _context.Orders.ToListAsync();
        }

        public async Task<Order?> GetOrderById(int id)

```



```
{  
    return await _context.Orders.FindAsync(id);  
}
```

```
public async Task<Order> CreateOrder(Order order)
```

```
{  
    // BUSINESS LOGIC  
    if (order.TotalAmount > 50000)  
        order.Status = OrderStatus.HighValue;  
    else  
        order.Status = OrderStatus.Pending;
```

```
    _context.Orders.Add(order);  
    await _context.SaveChangesAsync();
```

```
    return order;  
}
```

```
public async Task<bool> UpdateStatus(int id, string status)
```

```
{  
    var order = await _context.Orders.FindAsync(id);  
    if (order == null) return false;
```

```
    if (!Enum.TryParse<OrderStatus>(status, true, out var parsedStatus))  
        return false;
```

```
    order.Status = parsedStatus;  
    await _context.SaveChangesAsync();
```

```

        return true;
    }

    public async Task<bool> DeleteOrder(int id)
    {
        var order = await _context.Orders.FindAsync(id);
        if (order == null) return false;

        _context.Orders.Remove(order);
        await _context.SaveChangesAsync();

        return true;
    }
}

```

AUTOMAPPER :

```

using AutoMapper;
using OrderApi.DTOs;
using OrderApi.Models;

namespace OrderApi.Mapping
{
    public class MappingProfile : Profile
    {
        public MappingProfile()
        {
            CreateMap<OrderDto, Order>();
        }
    }
}

```

```
}
```

```
CONTROLLER :
```

```
using AutoMapper;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using OrderApi.DTOs;
```

```
using OrderApi.Models;
```

```
using OrderApi.Repositories;
```

```
namespace OrderApi.Controllers
```

```
{
```

```
    [Route("api/[controller]")]
```

```
    [ApiController]
```

```
    public class OrderController : ControllerBase
```

```
    {
```

```
        private readonly IOrderRepository _repo;
```

```
        private readonly IMapper _mapper;
```

```
        public OrderController(IOrderRepository repo, IMapper mapper)
```

```
        {
```

```
            _repo = repo;
```

```
            _mapper = mapper;
```

```
        }
```

```
        [HttpGet]
```

```
        public async Task<ActionResult> GetAll()
```

```
        {
```

```
            var orders = await _repo.GetAllOrders();
```

```
            return Ok(orders);
```

```
        }
```

```
[HttpGet("{id}")]
```

```
public async Task<IActionResult> GetById(int id)
```

```
{
```

```
    var order = await _repo.GetOrderById(id);
```

```
    if (order == null)
```

```
        return NotFound();
```

```
    return Ok(order);
```

```
}
```

```
[HttpPost]
```

```
public async Task<IActionResult> Create(OrderDto dto)
```

```
{
```

```
    if (string.IsNullOrEmpty(dto.CustomerName) || dto.TotalAmount <= 0)
```

```
        return BadRequest("Invalid data");
```

```
    var order = _mapper.Map<Order>(dto);
```

```
    var created = await _repo.CreateOrder(order);
```

```
    return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
```

```
}
```

```
[HttpPut("{id}")]
```

```
public async Task<IActionResult> UpdateStatus(int id, [FromBody] string status)
```

```
{
```

```
    var allowed = new[] { "Pending", "Completed", "Cancelled" };
```

```

        if (!allowed.Contains(status))
            return BadRequest("Invalid status");

        var updated = await _repo.UpdateStatus(id, status);

        if (!updated)
            return NotFound();

        return Ok("Status updated");
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> Delete(int id)
    {
        var deleted = await _repo.DeleteOrder(id);

        if (!deleted)
            return NotFound();

        return NoContent();
    }
}

```

MIDDLEWARE :

```
using System.Net;
```

```
using System.Text.Json;
```

```
namespace OrderApi.Middleware
```

```

{
    public class ExceptionMiddleware
    {
        private readonly RequestDelegate _next;

        public ExceptionMiddleware(RequestDelegate next)
        {
            _next = next;
        }

        public async Task Invoke(HttpContext context)
        {
            try
            {
                await _next(context);
            }
            catch (Exception)
            {
                context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;
                context.Response.ContentType = "application/json";

                var response = new { error = "Something went wrong" };
                await context.Response.WriteAsync(JsonSerializer.Serialize(response));
            }
        }
    }
}

```

BACKGROUND SERVICES:

```
using Microsoft.Extensions.Hosting;
```

```

namespace OrderApi.Services
{
    public class OrderBackgroundService : BackgroundService
    {
        protected override async Task ExecuteAsync(CancellationToken stoppingToken)
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                Console.WriteLine("Checking pending orders...");
                await Task.Delay(10000, stoppingToken);
            }
        }
    }
}

```

PROGRAM.CS :

```

using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Mapping;
using OrderApi.Middleware;
using OrderApi.Repositories;
using OrderApi.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

```

```
// InMemory DB
builder.Services.AddDbContext<AppDbContext>{opt =>
    opt.UseInMemoryDatabase("OrderDb")};

// Repository
builder.Services.AddScoped<IOrderRepository, OrderRepository>();

// AutoMapper
builder.Services.AddAutoMapper(typeof(MappingProfile));

// Background Service
builder.Services.AddHostedService<OrderBackgroundService>();

var app = builder.Build();

// Middleware
app.UseMiddleware<ExceptionMiddleware>();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseAuthorization();

app.MapControllers();

app.Run();
```


Date:-26/03/2026

=====

1st CODE –

```
class shape {
```

```
    int length;
```

```
    int width;
```

```
    shape(int l, int w) {
```

```
        length = l;
```

```
        width = w;
```

```
    }
```

```
}
```

```
class rectangle extends shape {
```

```
    rectangle(int l, int w) {
```

```
        super(l, w);
```

```
    }
```

```
    int calculatearea() {
```

```
        return length * width;
```

```
    }
```

```
    int calculateperimeter() {
```

```

        return 2 * (length + width);
    }
}

public class main {
    public static void main(String[] args) {
        java.util.Scanner sc = new java.util.Scanner(System.in);

        int length = sc.nextInt();
        int width = sc.nextInt();

        rectangle r = new rectangle(length, width);

        System.out.println("Area: " + r.calculatearea());
        System.out.println("Perimeter: " + r.calculateperimeter());
    }
}

```

2nd CODE –

```

import java.util.*;

class product {
    String name;
    double price;
    int quantity;

    product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }
}

```

```
}
```

```
String getproductdetails() {  
    return name + " - Price: " + price + ", Quantity: " + quantity;  
}  
}
```

```
class electronics extends product {  
    int warranty;  
  
    electronics(String name, double price, int quantity, int warranty) {  
        super(name, price, quantity);  
        this.warranty = warranty;  
    }  
}
```

```
String getproductdetails() {  
    return name + " - Price: " + price + ", Quantity: " + quantity +  
        ", Warranty: " + warranty + " months";  
}  
}
```

```
class clothing extends product {  
    String size;  
  
    clothing(String name, double price, int quantity, String size) {  
        super(name, price, quantity);  
        this.size = size;  
    }  
}
```

```

String getproductdetails() {
    return name + " - Price: " + price + ", Quantity: " + quantity +
        ", Size: " + size;
}
}

```

```

class inventory {
    List<product> list = new ArrayList<>();

    void addproduct(product p) {
        list.add(p);
        System.out.println("Product added to inventory: " + p.name);
    }

    void displayinventory() {
        System.out.println("\nInventory:");
        for (product p : list) {
            System.out.println(p.getproductdetails());
        }
    }

    void calculatetotalvalue() {
        double total = 0;
        for (product p : list) {
            total += p.price * p.quantity;
        }
        System.out.printf("\nTotal value of the inventory: %.2f", total);
    }
}

```

```
public class main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int num = Integer.parseInt(sc.nextLine());  
  
        inventory inv = new inventory();  
  
        for (int i = 0; i < num; i++) {  
            String line = sc.nextLine();  
            String[] parts = line.split(" ");  
  
            if (parts[0].equals("Electronics")) {  
                String name = parts[1];  
                double price = Double.parseDouble(parts[2]);  
                int quantity = Integer.parseInt(parts[3]);  
                int warranty = Integer.parseInt(parts[4]);  
  
                inv.addproduct(new electronics(name, price, quantity, warranty));  
  
            } else if (parts[0].equals("Clothing")) {  
                String name = parts[1];  
                double price = Double.parseDouble(parts[2]);  
                int quantity = Integer.parseInt(parts[3]);  
                String size = parts[4];  
  
                inv.addproduct(new clothing(name, price, quantity, size));  
            }  
        }  
    }  
}
```

```
        inv.displayinventory();  
        inv.calculatetotalvalue();  
    }  
}
```

3rd CODE – VIRUS RESOLVING :

```
import java.util.*;
```

```
public class main {
```

```
    public static int virusResolving(int n, int x, int y, int[] damages) {
```

```
        // WRITE YOUR LOGIC HERE
```

```
        if (x < 2 * y) {  
            int pairs = n / 2;  
            int remaining = n % 2;  
            return pairs * x + remaining * y;  
        } else {  
            return n * y;  
        }  
    }  
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();
```

```
    int x = sc.nextInt();
```

```
    int y = sc.nextInt();
```

```
int[] damages = new int[n];  
for (int i = 0; i < n; i++) {  
    damages[i] = sc.nextInt();  
}  
  
int result = virusResolving(n, x, y, damages);  
System.out.println(result);  
}  
}
```

Date:-27/03/2026

=====

1st CODE – Model :

```
public class scoremodel
{
    public string name { get; set; }
    public int marks { get; set; }
}
```

CONTROLLER :

```
using Microsoft.AspNetCore.Mvc;
```

```
public class homecontroller : Controller
```

```
{
    public IActionResult index()
    {
        return View();
    }
}
```

```
[HttpPost]
```

```
public IActionResult index(scoremodel model)
{
    return View(model);
}
}
```


VIEW :

@model scoremodel

```
<form method="post">
    <label>Name:</label>
    <input type="text" name="name" required />
    <br />

    <label>Marks:</label>
    <input type="number" name="marks" required />
    <br />

    <button type="submit">Submit</button>
</form>
```

```
@if (Model != null)
{
    if (Model.marks >= 50)
    {
        <h2 style="color:green;">Pass</h2>
    }
    else
    {
        <h2 style="color:red;">Fail</h2>
    }
}
```

2nd CODE – CUSTOM MIDDLEWARE CLASS :

```
using System.Diagnostics;
using System.Threading.Tasks;
```

```
using Microsoft.AspNetCore.Http;
```

```
public class requesttimingmiddleware
```

```
{
```

```
    private readonly RequestDelegate next;
```

```
    public requesttimingmiddleware(RequestDelegate next)
```

```
    {
```

```
        this.next = next;
```

```
    }
```

```
    public async Task Invoke(HttpContext context)
```

```
    {
```

```
        var stopwatch = Stopwatch.StartNew();
```

```
        await next(context); // call next middleware
```

```
        stopwatch.Stop();
```

```
        var time = stopwatch.ElapsedMilliseconds;
```

```
        context.Response.Headers["X-Response-Time"] = time + "ms";
```

```
    }
```

```
}
```

```
REGISTER MIDDLEWARE :
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
var app = builder.Build();
```

```
app.UseMiddleware<requesttimingmiddleware>();
```

```
app.MapGet("/", () => "Hello World!");
```

```
app.Run();
```

3rd CODE – Employee :

```
public class employee
{
    public int id { get; set; }
    public string name { get; set; }
    public double salary { get; set; }
    public string department { get; set; }
}
```

LINQ QUERY :

```
var result = context.employees
    .select(e => new
    {
        e.name,
        e.department
    })
    .tolist();
```

CONTROLLER :

```
public IActionResult getemployees()
{
    var data = context.employees
        .select(e => new
        {
            e.name,
            e.department
        })
}
```

```
.tolist();
```

```
return Json(data);
```

```
}
```

4th CODE –

```
public class product
```

```
{
```

```
    public string name { get; set; }
```

```
    public double price { get; set; }
```

```
}
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using System.Linq;
```

```
using System.Collections.Generic;
```

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class productscontroller : ControllerBase
```

```
{
```

```
    private static List<product> products = new List<product>
```

```
    {
```

```
        new product { name = "Laptop", price = 800 },
```

```
        new product { name = "Mouse", price = 20 },
```

```
        new product { name = "Phone", price = 500 },
```

```
        new product { name = "Keyboard", price = 100 }
```

```
    };
```

```
[HttpGet]
```

```
public IActionResult getproducts(double? minPrice, double? maxPrice)
```

```
{
```

```

var query = products.AsQueryable();

// Conditional filtering
if (minPrice.HasValue)
{
    query = query.Where(p => p.price >= minPrice.Value);
}

if (maxPrice.HasValue)
{
    query = query.Where(p => p.price <= maxPrice.Value);
}

return Ok(query.ToList());
}
}

```

5th CODE –

```

using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        int count = 0;

        // check if cookie exists
        if (Request.Cookies["visitcount"] != null)
        {
            count = int.Parse(Request.Cookies["visitcount"]);

```

```

    }

    // increment count
    count++;

    // set/update cookie
    Response.Cookies.Append("visitcount", count.ToString(), new CookieOptions
    {
        Expires = DateTime.Now.AddDays(7)
    });

    ViewBag.count = count;

    return View();
}
}

```

PART -2 :

1ST CODE :

```

public class orderform
{
    public string productname { get; set; }

    public int quantity { get; set; }

    // optional field
    public string? discountcode { get; set; }
}

using Microsoft.AspNetCore.Mvc;

```

```
public class ordercontroller : Controller
{
    [HttpGet]
    public IActionResult index()
    {
        return View(new orderform());
    }

    [HttpPost]
    public IActionResult index(orderform model)
    {
        // if quantity < 5 → discount should be ignored
        if (model.quantity < 5)
        {
            // remove validation error (if any)
            ModelState.Remove("discountcode");

            // optional: keep it null
            model.discountcode = null;
        }

        if (ModelState.IsValid)
        {
            ViewBag.message = "Order submitted successfully";
        }

        return View(model);
    }
}
```

VIEW :

@model orderform

```
<form method="post">
```

```
    <label>Product Name:</label>
```

```
    <input type="text" asp-for="productname" />
```

```
    <br />
```

```
    <label>Quantity:</label>
```

```
    <input type="number" asp-for="quantity" />
```

```
    <br />
```

```
    @if (Model.quantity >= 5)
```

```
    {
```

```
        <label>Discount Code:</label>
```

```
        <input type="text" asp-for="discountcode" />
```

```
        <br />
```

```
    }
```

```
    <button type="submit">Submit</button>
```

```
</form>
```

```
@if (ViewBag.message != null)
```

```
{
```

```
    <h3>@ViewBag.message</h3>
```

```
}
```

2nd CODE –

```
using System.IO;
```



```
using System.Text;

using System.Threading.Tasks;

using Microsoft.AspNetCore.Http;

public class responsemodifymiddleware
{
    private readonly RequestDelegate next;

    public responsemodifymiddleware(RequestDelegate next)
    {
        this.next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        // store original response stream
        var originalBody = context.Response.Body;

        using (var memoryStream = new MemoryStream())
        {
            // replace response stream with memory stream
            context.Response.Body = memoryStream;

            // call next middleware / controller
            await next(context);

            // read response from memory stream
            memoryStream.Seek(0, SeekOrigin.Begin);
            string responseBody = await new StreamReader(memoryStream).ReadToEndAsync();
```

```

// modify response
if (responseBody.Contains("Hello"))
{
    responseBody = responseBody.Replace("Hello", "Hello - Processed by Middleware");
}

// write modified response back
var modifiedBytes = Encoding.UTF8.GetBytes(responseBody);

context.Response.Body = originalBody;
await context.Response.Body.WriteAsync(modifiedBytes, 0, modifiedBytes.Length);
}
}
}

```

REGISTER MIDDLEWARE :

```

var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseMiddleware<responsemodifymiddleware>();

app.MapGet("/", () => "Hello World!");

```

```
app.Run();
```

3rd CODE :

```

public class product
{
    public string name { get; set; }
    public double price { get; set; }
}

```

```

        public string category { get; set; }
    }

    using Microsoft.AspNetCore.Mvc;
    using System.Linq;

    [ApiController]
    [Route("api/[controller]")]
    public class productscontroller : ControllerBase
    {
        private readonly AppDbContext context;

        public productscontroller(AppDbContext context)
        {
            this.context = context;
        }

        [HttpGet]
        public IActionResult getproducts(double? minPrice, string category, string search)
        {
            // IMPORTANT: IQueryable (not List)
            var query = context.products.AsQueryable();

            // Apply filters dynamically

            if (minPrice.HasValue)
            {
                query = query.Where(p => p.price >= minPrice.Value);
            }
        }
    }

```

```
if (!string.IsNullOrEmpty(category))
{
    query = query.Where(p => p.category == category);
}

if (!string.IsNullOrEmpty(search))
{
    query = query.Where(p => p.name.Contains(search));
}

// SINGLE DB EXECUTION happens here
var result = query.ToList();

return Ok(result);
}
}
```